香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)
信息枢纽
INFORMATION HUB
人工智能学域
ARTIFICIAL INTELLIGENCE

Robotic Control

Course AIAA 4220

Week8- Lecture 16

Changhao Chen

Assistant Professor

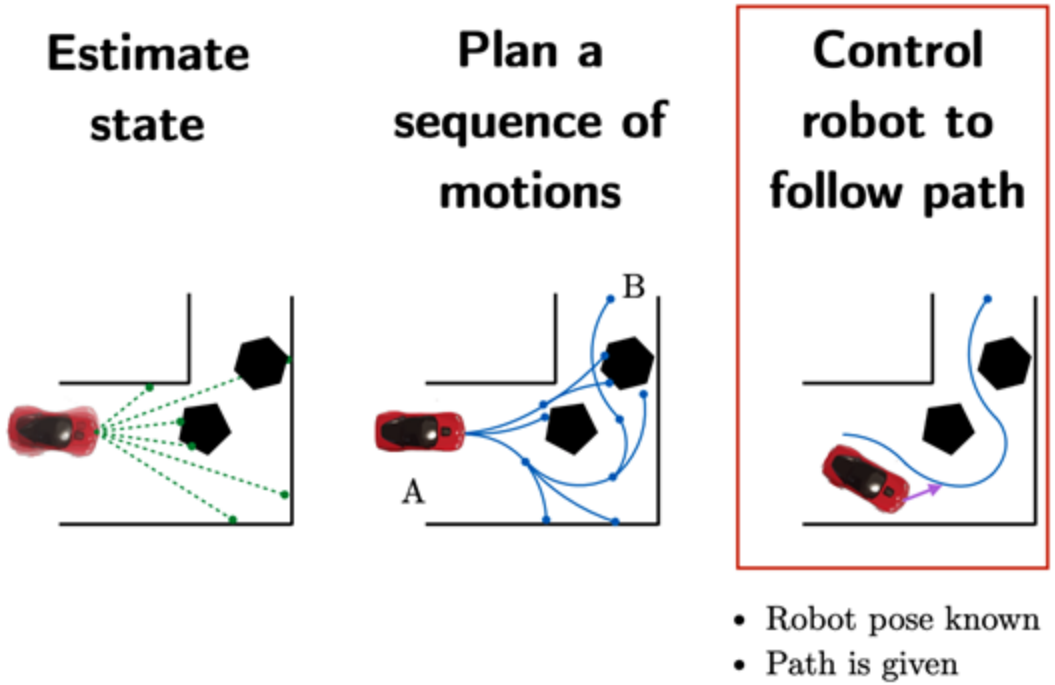
HKUST (GZ)

Robotic Control

What Is Control?

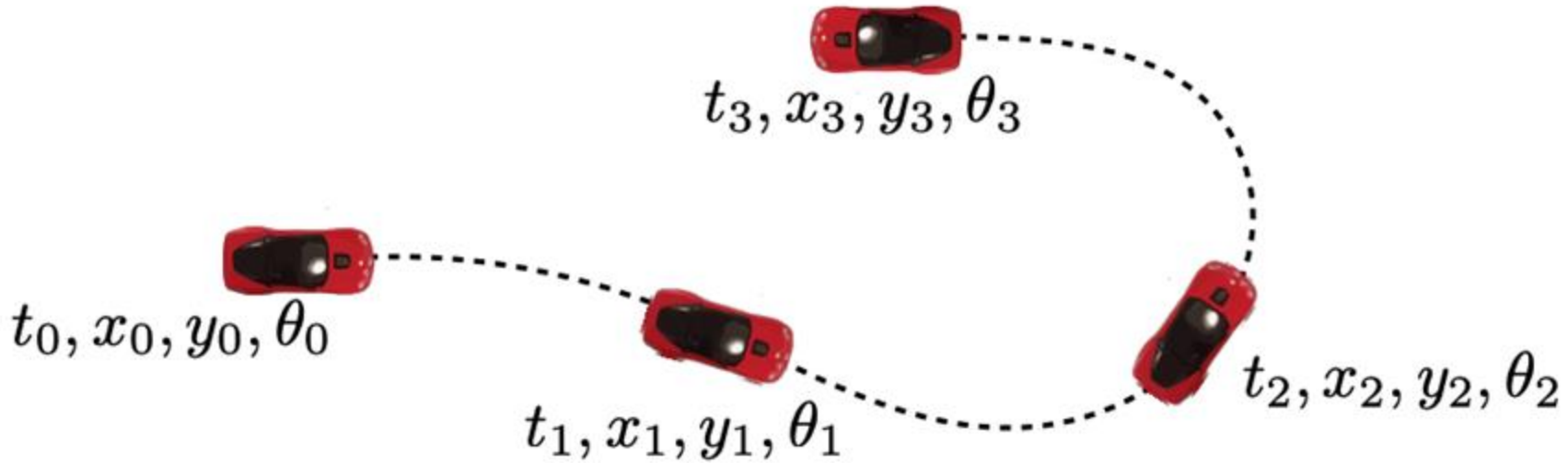
•Definition: **Ensuring the robot follows planned motion despite disturbances and model errors.**

•Role in embodied AI: Bridge between planning and actuation.



The control framework

- Target: we want to get the car to reach a sequence of poses



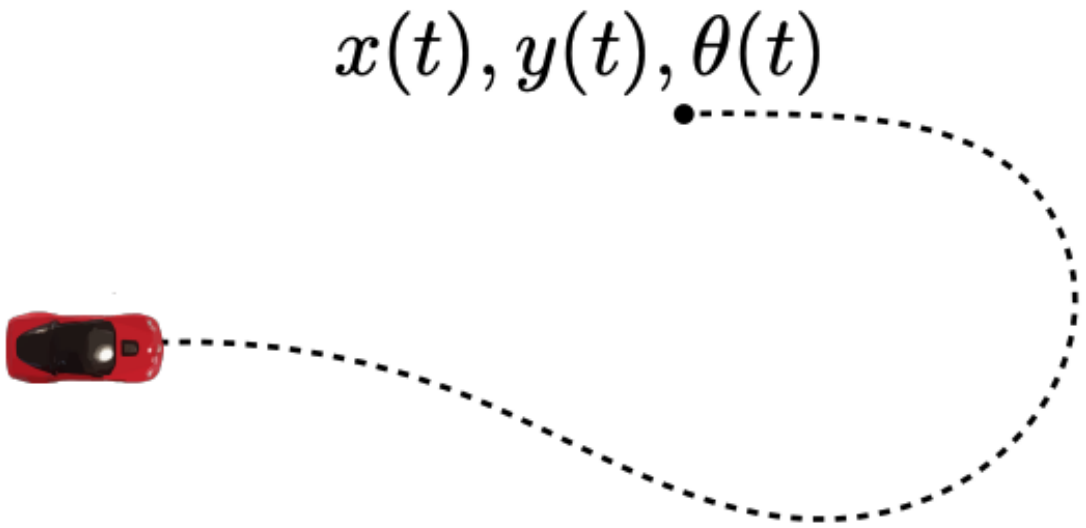
This example is for a planar robot with poses defined in SE(2). The trajectory comes from the planning module

Can express this as wanting to track a reference *trajectory*

$$x(t), y(t), \theta(t)$$

The control framework

- Let's say we want to track a **reference trajectory**



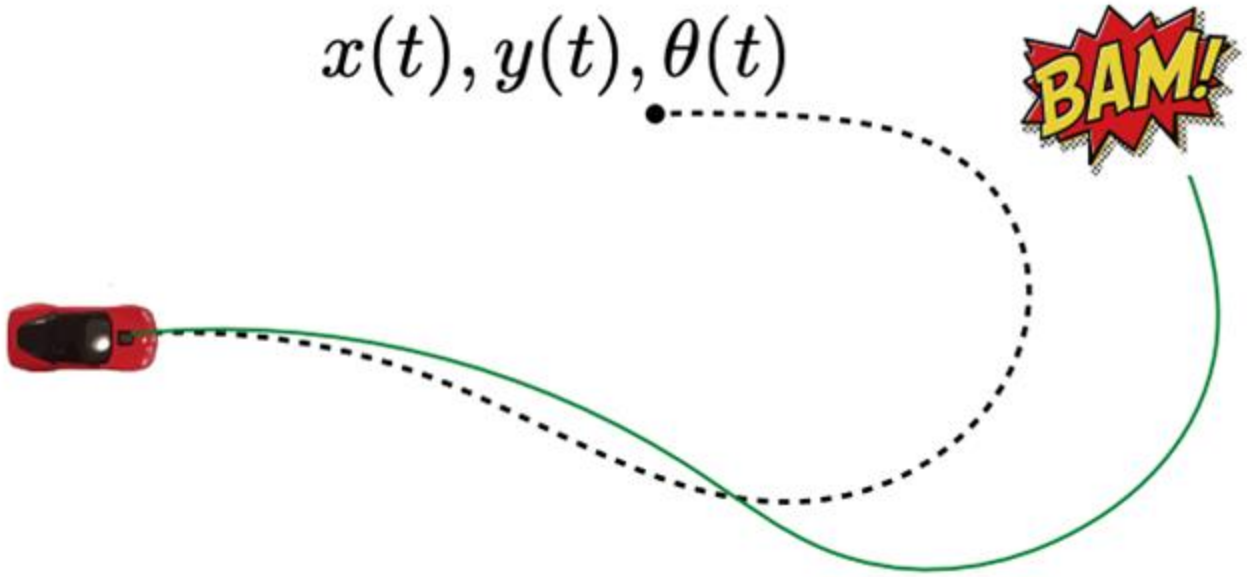
Given time step t, we have a reference pose

Objective: Figure out a **control** trajectory $u(t)$ to achieve this

In our case, we will focus on steering angle $\delta(t)$ as control input

Why do we need feedback?

- Let's say we want to track a reference trajectory



What if we send out steering angles $\delta(t)$ obtained from kinematic car model?

This means given the reference trajectory, we directly convert that to delta of t using the vehicle model. This **open loop control** will lead to large errors.

Open loop control: the controller is not affected by the state or any **feedback**.

Example: washing machine timer.

Why do we need feedback?

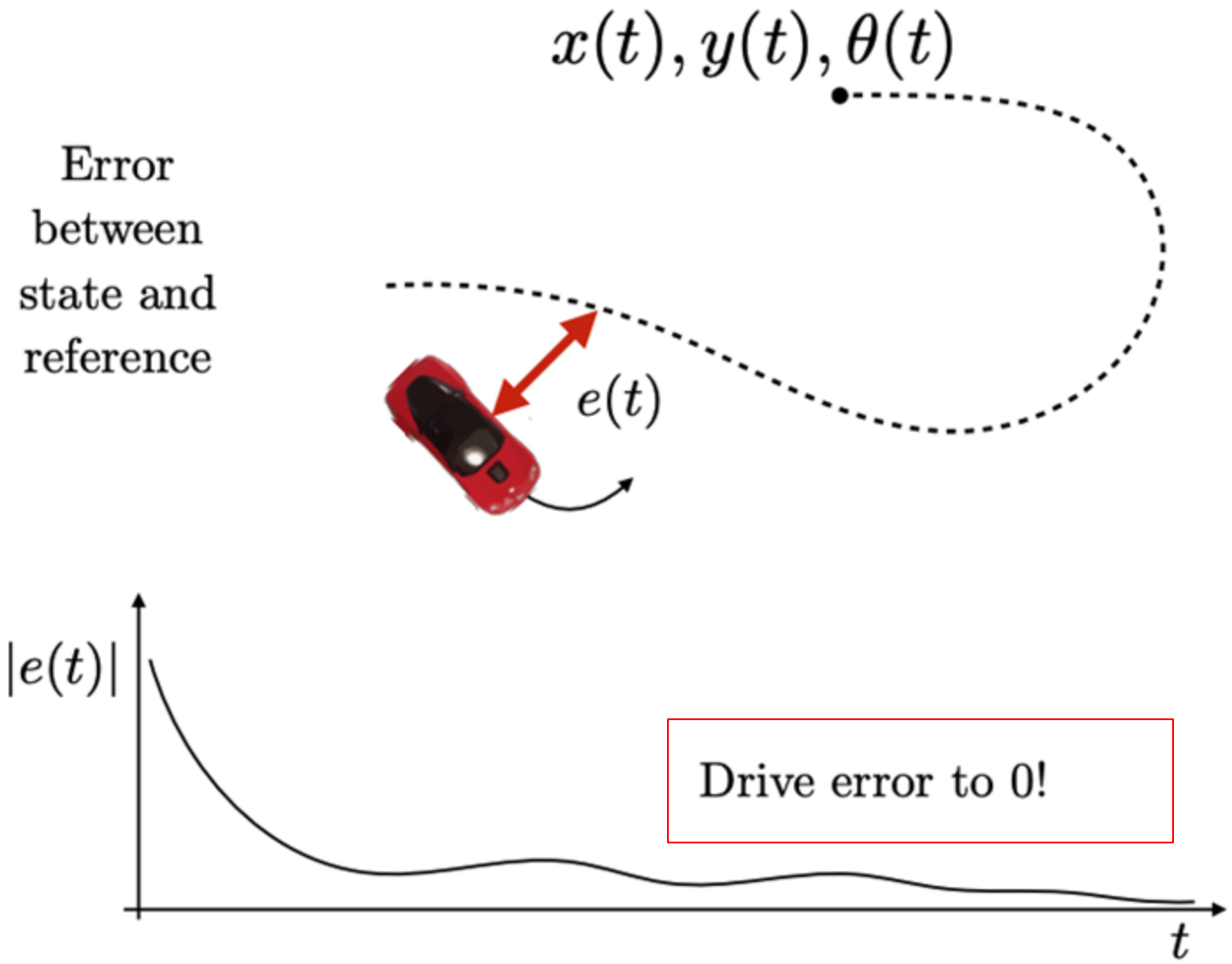
- Principle of feedback control:
  - Measure error between reference and state at all times
  - Minimize this error

Feedback Control

•Open-loop vs. closed-loop
•Feedback: compare measured output vs. desired reference
•Example: temperature control or car cruise control

Feedback Control

Measure error and minimize it



Robotic Control

- Industrial robots at work

These robots are usually assumed “fully actuated” and there is little environmental disturbance

Robotic Control

Assumptions made by such controllers

1. **Fully actuated**: There exists an inverse mapping from reference to control actions

$$\sigma(t) \rightarrow u(t)$$

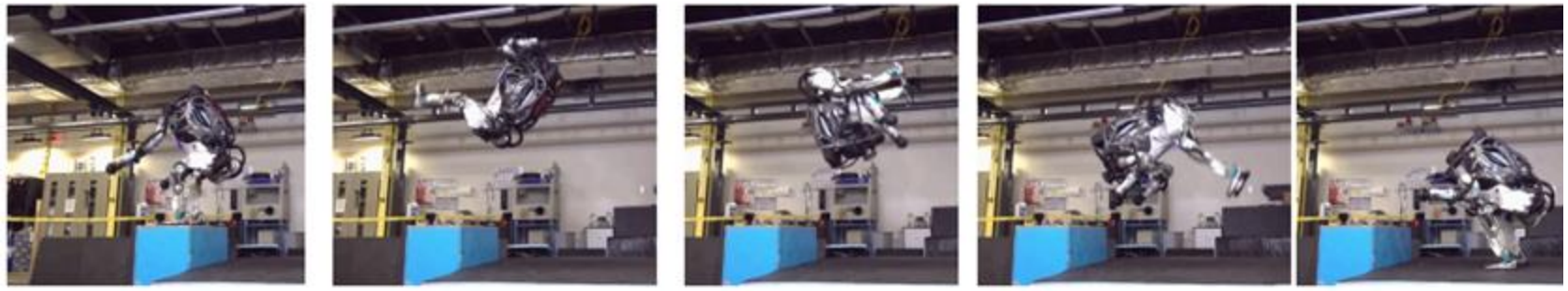
2. Almost no execution error or state estimation error

3. Enough control authority to clamp down errors / overcome disturbances
   No external influence factors

Robotic Control

Instead, what are harder problems for control?

- In real-world, there are many underactuated systems
- What affects the error between robot state and reference?



Some initial motor thrust ...

Whole lot of gravity!

Whole lot of momentum!

... some precise control adjustments

Robotic Control

Other challenges for mobile robot control:

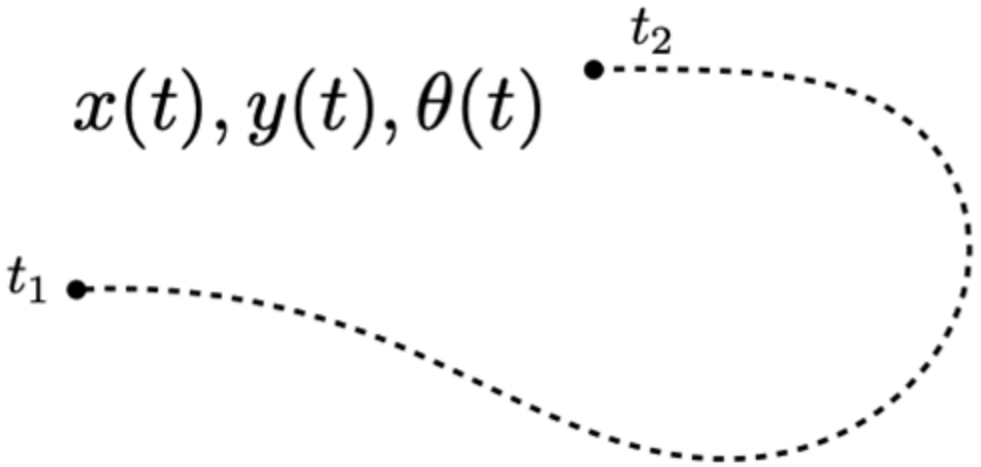
1. Unexpected obstacle avoidance
2. Noisy state estimation
3. Motion model changing on the fly (drifting race cars, etc.)

A generic template for a controller:

1. Get a reference path / trajectory to track
2. Pick a point on the reference
3. Compute error to reference point
4. Compute control law to minimize error

Step 1: Get a reference

- Reference can be a time-parameterized trajectory



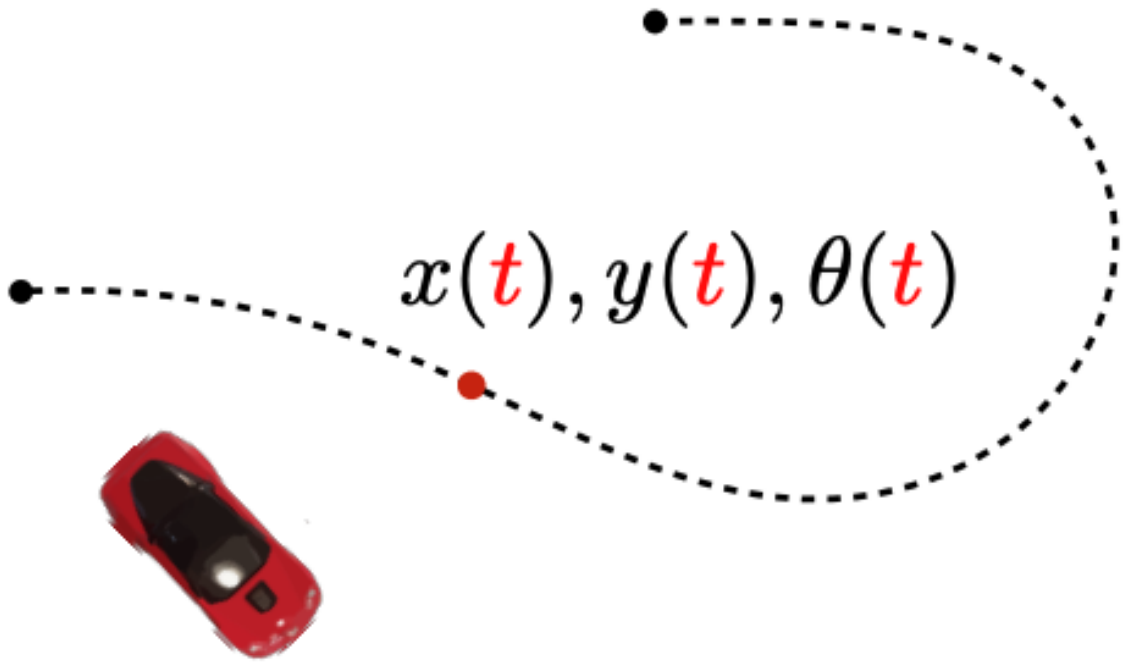
The time index refers to a desired pose we want the system to achieve at a given time

Pro: Useful if we want the robot to respect time constraints

Con: Sometimes we care only about deviation from reference

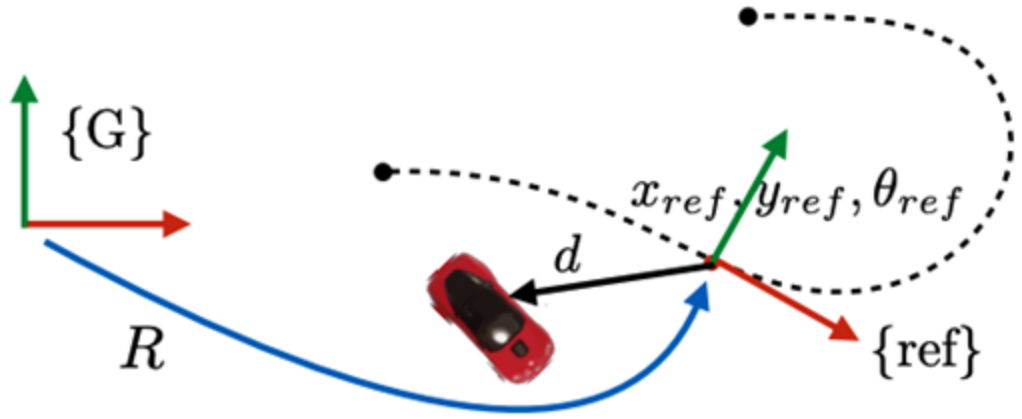
Step 2: Pick a point on the reference

- Pick a reference point on the trajectory / path



If we are using a time-parameterized trajectory, the current time is the natural reference

Step 3: Compute error to reference point



1. Define error vector

$$d = \begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} x_{ref} \\ y_{ref} \end{bmatrix}$$

Translation error

2. Rotate error vector to be in the **reference** frame

Rotation Matrix

$$e = R^T(\theta_{ref}) \left(\begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} x_{ref} \\ y_{ref} \end{bmatrix} \right)$$

$$e = \begin{bmatrix} \cos(\theta_{ref}) & \sin(\theta_{ref}) \\ -\sin(\theta_{ref}) & \cos(\theta_{ref}) \end{bmatrix} \left(\begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} x_{ref} \\ y_{ref} \end{bmatrix} \right)$$

Translate the point to the reference point first, then compute the rotation

Robotic Control

Step 4: Compute control law

- Compute control action based on instantaneous error

$$u = K(\mathbf{x}, e)$$

control — (steering angle, speed)

state — $\mathbf{x}$

error — e

Compute the new control action based on the kinematics model (not covered in our course),
Apply action, robot moves a bit,
Then compute new error, repeat

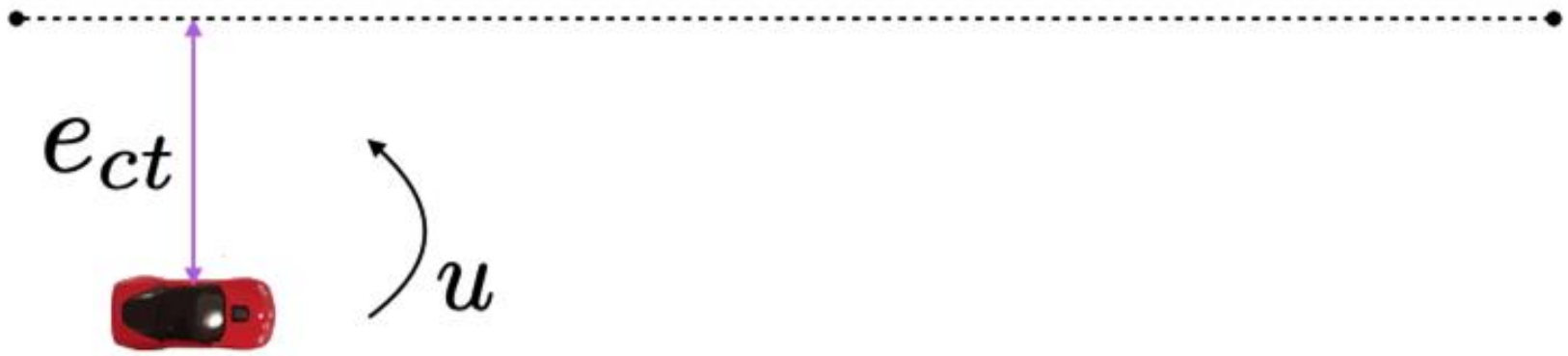
Different Controllers

- **Proportional–integral–derivative (PID) controller**
- Pure-pursuit control
- Linear Quadratic Regulator (LQR)
- **Model Predictive Control (MPC)**

PID Controller

Proportional–integral–derivative (PID)

- Goal: Select a control law that tries to drive error to zero (and keep it there)



"e_{ct}" refers to "cross-track error": the lateral distance between the robot's current position and the closest point in the reference path

PID Controller

Control input u(t) is proportional to **error**, its **accumulation**, and its **rate of change**.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\,d\tau + K_d \frac{de(t)}{dt}$$

Where:

- $e(t) = r(t) - y(t)$: tracking error
- K_p: proportional gain
- K_i: integral gain
- K_d: derivative gain

PID Controller

Term	Effect	Advantage	Risk
P (Proportional)	Responds to current error	Fast correction	Can overshoot
I (Integral)	Accumulates past error	Eliminates steady-state bias	May cause oscillation
D (Derivative)	Predicts future error	Improves stability	Sensitive to noise

For Example- Driving a car:

- P: Steer based on how far off you are now
- I: Adjust for consistent drift
- D: Smooth reaction before overshooting

Model Predictive Control (MPC)

At each time step, **predict future behavior** using a system model and **optimize future control inputs** to minimize a cost function.

$$\min_{u_{0:H}} \sum_{k=0}^{H} \left[(x_k - x_{\text{goal}})^T Q (x_k - x_{\text{goal}}) + u_k^T R u_k \right]$$

- Q: penalty for state error (tracking error)
- R: penalty for control effort
- Balances **accuracy vs smooth control**

MPC finds control inputs that track target while minimizing effort and respecting constraints.

MPC Control Loop

1. Measure state x_t
2. **Predict future states** over horizon H
   using model $x_{t+1} = f(x_t, u_t)$
3. **Solve optimization**

$$\min_{u_{0:H}} \sum_{k=0}^{H} \ell(x_k, u_k)$$

4. **Apply first control action**
5. **Repeat at next timestep**

Example – Self-Driving Car Steering

Goal: Follow centerline of lane.
System: Vehicle kinematics model
Prediction horizon: 2–3 seconds
Control variables: steering angle, velocity
Cost function: lateral deviation + control effort

Other Nonlinear Control

Many embodied systems (drones, manipulators) are nonlinear

Techniques:
•Feedback linearization
•Sliding mode control
•Adaptive control

Learning-Based Control
•**End-to-end policy learning:** map sensory input → control commands
•**Model-based RL:** learn dynamics + use MPC (e.g., PILCO, MBPO)
•**Policy gradient methods:** train via reward signals
•**Safe RL / Imitation-based control:** constrain exploration

Integration of Planning & Control

- Planning gives a **path or trajectory**, assuming ideal execution.
- Control executes the **commands**, but real systems face **disturbances, delays, and uncertainty**.

Goal:

- Create a **closed-loop system** that continuously adjusts plans and control inputs as the robot interacts with the real world.

Two Levels of Integration

1) Hierarchical Integration

- **Global Planner:** Computes long-term route (map-level).
- **Local Planner / Controller:** Tracks and adjusts for local obstacles, dynamics, and uncertainty.
- Used in most **robot navigation stacks (e.g., ROS)**.

2) Tight / Receding Horizon Integration

- **Model Predictive Control (MPC)** naturally integrates both:
  - Predicts motion over a horizon → acts as short-term planner.
  - Optimizes control inputs → acts as controller.
- Receding horizon → new plan every time step = *online planning + control*.

Applications in Embodied AI

1) Autonomous Driving

- Global route planning (e.g., HD maps)
- Local trajectory optimization (e.g., MPC)
- Low-level control (e.g., PID / LQR)
- Real-time feedback integration via sensors (e.g., LiDAR, cameras)

2) Mobile Robots

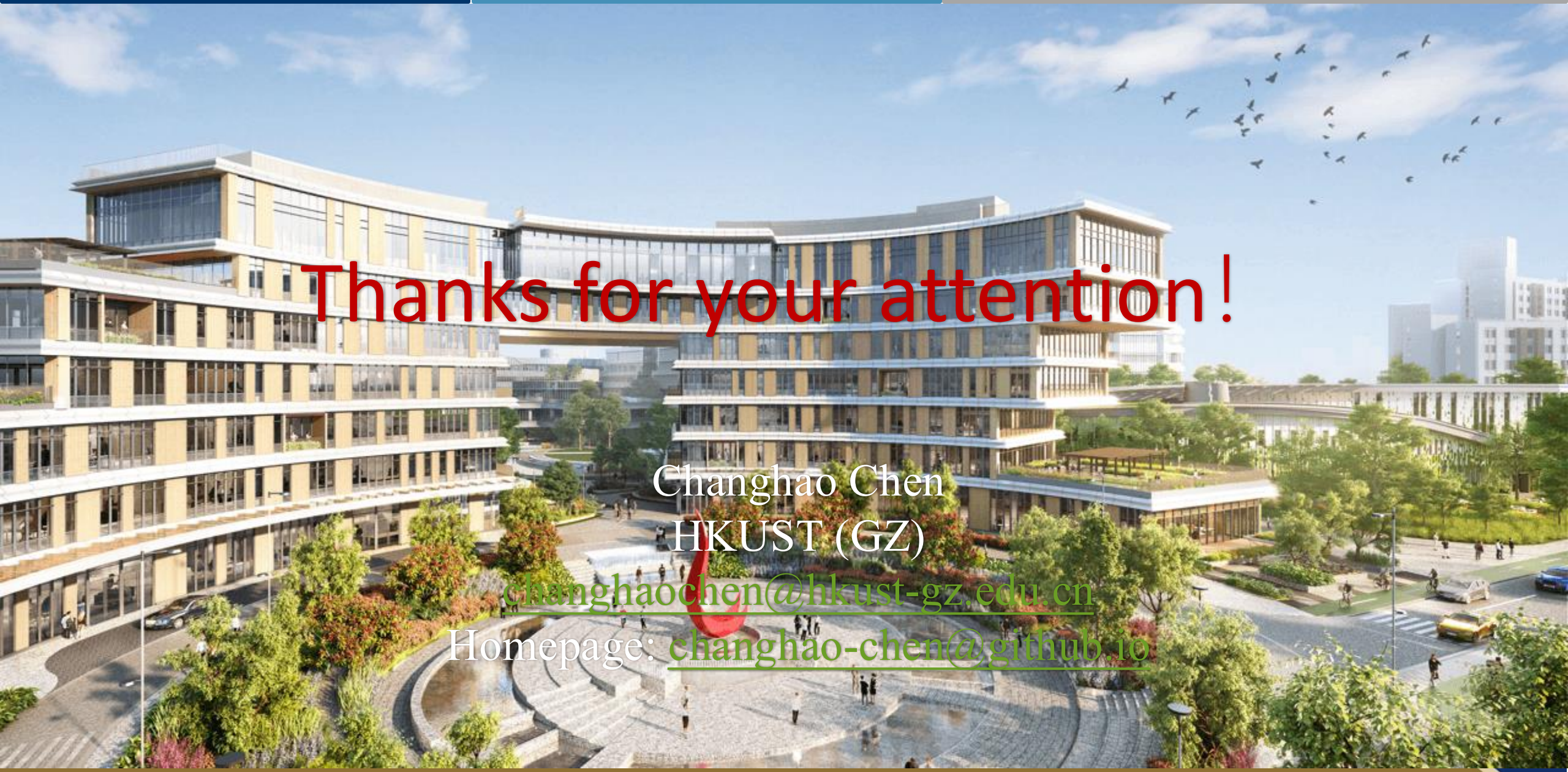
- ROS navigation: e.g., A* + DWA + PID
- Replanning when unexpected obstacles appear
- Warehouse robots or delivery bots

3) Aerial Drones

- Global waypoint planner (e.g., RRT*)
- Trajectory optimization (e.g., minimum-snap)
- MPC or neural controllers for flight stability
- Visual-inertial feedback for closed-loop control

4) Legged Robots

- Footstep planning + dynamic balance control
- Whole-body MPC for predictive gait adaptation



Thanks for your attention!

Changhao Chen
HKUST (GZ)
changhaochen@hkust-gz.edu.cn
Homepage: changhao-chen@github.io